

002 Ingress Reference Patterns

 Page Status - Ready for Review

Description

Table of Contents

- [Description](#)
- [Table of Contents](#)
- [What is an Ingress Reference Pattern?](#)
- [Principles of Ingress Reference Pattern](#)
- [Ingress reference patterns](#)
 - [Anti-Patterns](#)
 - [Producer-Consumer](#)
 - [Event Streams Connector](#)
 - [Event Mediator](#)
 - [Event Choreography](#)
 - [Managed File Transfer \(MFT\) / Electronic Data Interface \(EDI\)](#)

What is an Ingress Reference Pattern?

Ingress reference patterns refer to how data from outside a system is delivered. We have identified six (6) defining principles for measuring an ingress pattern:

Principles of Ingress Reference Pattern

Principle Description

- **Reliability:** Always do what the consumer expects
- **Availability:** Always expect the consumer needs without interrupting the experience
- **Simplicity:** Always design architectural patterns that are simple and extendable
- **Observability:** Always have the ability to trace audit and heal
- **Extensibility:** Always design platform independent patterns
- **Maintainability:** Always design with the capability to add new features with no impact

The intention is define common or most referenced ingress patterns. Identification of the type of ingress patterns for a subject piece of information provides benefits for architectural design, observability, quality, latency and consumer satisfaction.

Ingress reference patterns

The list of common ingress reference patterns are defined below with current included implementations.

Anti-Patterns

An anti-pattern is recognized as a provided solution to a problem that generates negative results. This is a pattern, when identified, that should be avoided and an ingress pattern which fulfills the ingress principles suggested or implemented.

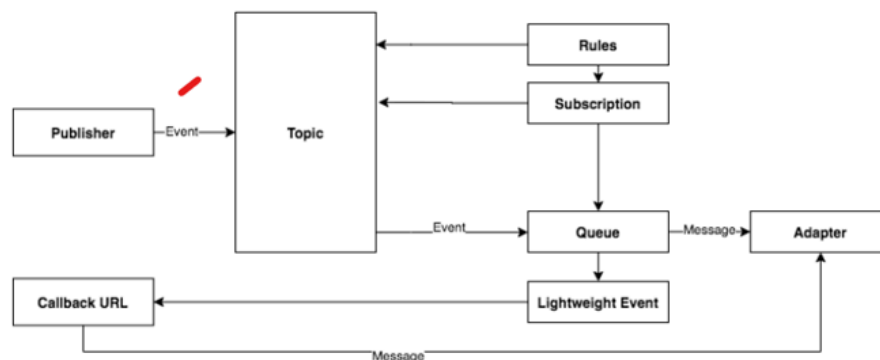
Examples of anti-patterns: Reverse Events: The notification of action prior to the occurrence or events based on SLA schedules.

- On-Demand Pull: Ask the producer to say what is ready, do not assume.
- Smart Pipe, Dumb End Point: Putting logic within the pipeline and service bus.

Make the data simple.

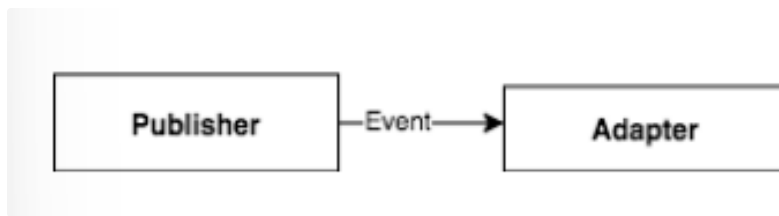
Producer-Consumer

1. Producer generates events describing business value.
2. Consumer subscribes to business value event
3. Consumer applies applicable filter(s) to event.
4. Payload >10mb, use lightweight model (callback url to smart endpoint)



Event Streams Connector

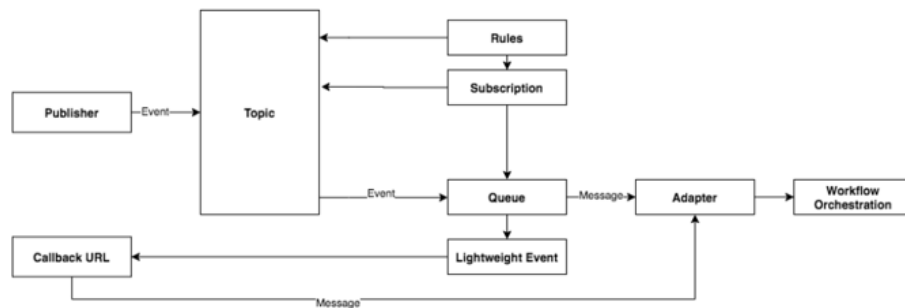
1. Producer pushes notifications as occurred. Does not wait for consumer request
2. Consumer responds immediately
3. Notification is "fire and forget"
4. Notification does not specific customer action
5. Multiple deliveries to customer



Event Mediator

Patterns in this category address the need to handle synchronization, transformation and orchestration of chunks of data to ensure systems always contain timely and meaningful data.

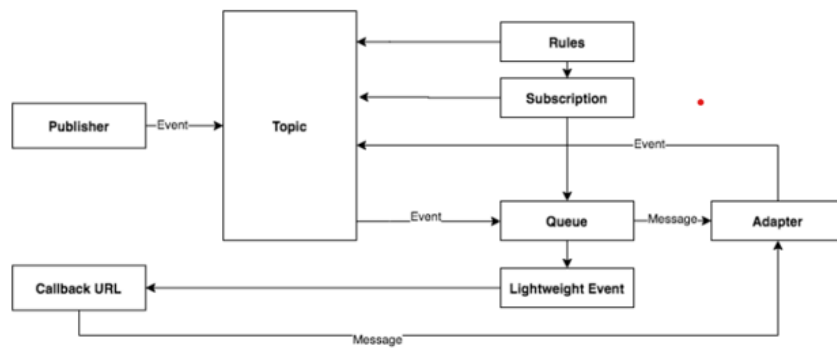
Mediator pattern is a good option if you know that services need to be sequenced, prioritized and run with a set of pre-defined steps. If sequence of steps has to be strictly followed then Mediator is the good option. It still can be loosely coupled and delegate their interaction to a mediator object instead of interacting with each other directly.



Event Choreography

Event driven solutions fit well with this pattern especially when decoupled services are self-contained without a central brain or conductor. Services collaborate by publishing and subscribing to messages and events. It is the reactive coordination of component actions based on events in a distributed system.

It's self-contained and decentralized group of services that does not depend on each other and can act on its own; publishing and subscribing to messages and events.



Managed File Transfer (MFT) / Electronic Data Interface (EDI)

One application writes a file that the other application reads and vice versa. The file format, standards, location, privileges and read/write coordination must be negotiated between the two applications.

